UNIT II : Analytical Questions

Slot C

Name : K. Lahari **Reg No. : 192325118**
Course : Cryptography And Network Security **Course Code : CSA5104**

Question 1

Perform DES encryption for the following plaintext and key, showing the Initial Permutation, generation of the 16 round keys, the Round 1 Feistel operation, the Final Permutation, and the resulting ciphertext.

Plaintext (64-bit, hex): 0123456789ABCDEF

Key (64-bit, hex): 133457799BBCDFF1

Answer

Step 1: Initial Permutation (IP)

The 64-bit plaintext is rearranged according to the standard DES IP table, producing:

IP output: CC00CCFFF0AAF0AA

L0 (left 32 bits): CC00CCFF

R0 (right 32 bits): F0AAF0AA

Step 2: Generation of the 16 Round Keys

The 64-bit key is first reduced to 56 bits via Permuted Choice 1 (PC-1), split into two 28-bit halves C0 and D0. Each half is left-circular-shifted by a fixed schedule (1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1 bits for rounds 1-16), and Permuted Choice 2 (PC-2) extracts the 48-bit round key K(i) from the concatenation of Ci and Di at each round.

Round	Round Key K(i) — 48 bits (hex)
1	1B02EFFC7072
2	79AED9DBC9E5
3	55FC8A42CF99
4	72ADD6DB351D
5	7CEC07EB53A8
6	63A53E507B2F
7	EC84B7F618BC
8	F78A3AC13BFB
9	E0DBEBEDE781
10	B1F347BA464F
11	215FD3DED386
12	7571F59467E9
13	97C5D1FABA41
14	5F43B7F2E73A
15	BF918D3D3F0A
16	CB3D8B0E17F5

Step 3: Round 1 Feistel Operation

Round 1 takes L0 and R0 from the IP output and round key K1:

- R0 is expanded from 32 to 48 bits via the E-table: E(R0) = 7A15557A1555
- E(R0) is XORed with round key K1: E(R0) ⊕ K1 = 6117BA866527
- The 48-bit result is split into eight 6-bit blocks and passed through S-boxes S1-S8, producing a 32-bit output: 5C82B597

- The S-box output is permuted through the P-table, giving f(R0, K1) = 234AA9BB
- R1 = L0 ⊕ f(R0, K1) = CC00CCFF ⊕ 234AA9BB = EF4A6544
- L1 = R0 = F0AAF0AA

Result after Round 1: L1 = F0AAF0AA, R1 = EF4A6544

Step 4: Remaining Rounds (2-16)

The same Feistel procedure (expansion, XOR with round key, S-box substitution, P-permutation, XOR with the previous left half) is repeated for all 16 rounds, each using its own round key from Step 2. The resulting Li/Ri pairs are:

Round	**Li (hex)**	**Ri (hex)**
1	F0AAF0AA	EF4A6544
2	EF4A6544	CC017709
3	CC017709	A25C0BF4
4	A25C0BF4	77220045
5	77220045	8A4FA637
6	8A4FA637	E967CD69
7	E967CD69	064ABA10
8	064ABA10	D5694B90
9	D5694B90	247CC67A
10	247CC67A	B7D5D7B2
11	B7D5D7B2	C5783C78
12	C5783C78	75BD1858
13	75BD1858	18C3155A
14	18C3155A	C28C960D
15	C28C960D	43423234
16	43423234	0A4CD995

Step 5: Final Permutation (FP)

After round 16 there is no final swap of L16/R16; the pre-output block R16‖L16 is formed and passed through the FP table (the inverse of IP):

Pre-output block (R16 ‖ L16): 0A4CD99543423234

Ciphertext after FP: 85E813540F0AB405

Final Answer — Ciphertext: 85E813540F0AB405

Question 2

Perform DES encryption for the following plaintext and key, showing the Initial Permutation, generation of the 16 round keys, the Round 1 Feistel operation, the Final Permutation, and the resulting ciphertext.

Plaintext (64-bit, hex): 1111222233334444

Key (64-bit, hex): AABB09182736CCDD

Answer

Step 1: Initial Permutation (IP)

The 64-bit plaintext is rearranged according to the standard DES IP table, producing:

IP output: C033C033003C003C

L0 (left 32 bits): C033C033

R0 (right 32 bits): 003C003C

Step 2: Generation of the 16 Round Keys

The 64-bit key is first reduced to 56 bits via Permuted Choice 1 (PC-1), split into two 28-bit halves C0 and D0. Each half is left-circular-shifted by a fixed schedule (1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1 bits for rounds 1-16), and Permuted Choice 2 (PC-2) extracts the 48-bit round key K(i) from the concatenation of Ci and Di at each round.

Round	**Round Key K(i) — 48 bits (hex)**
1	194CD072DE8C
2	4568581ABCCE
3	06EDA4ACF5B5
4	DA2D032B6EE3
5	69A629FEC913
6	C1948E87475E
7	708AD2DDB3C0
8	34F822F0C66D
9	84BB4473DCCC
10	02765708B5BF
11	6D5560AF7CA5
12	C2C1E96A4BF3
13	99C31397C91F
14	251B8BC717D0
15	3330C5D9A36D
16	181C5D75C66D

Step 3: Round 1 Feistel Operation

Round 1 takes L0 and R0 from the IP output and round key K1:

- R0 is expanded from 32 to 48 bits via the E-table: E(R0) = 0001F80001F8
- E(R0) is XORed with round key K1: E(R0) ⊕ K1 = 194D2872DF74
- The 48-bit result is split into eight 6-bit blocks and passed through S-boxes S1-S8, producing a 32-bit output: 122CEF3A
- The S-box output is permuted through the P-table, giving f(R0, K1) = 5B2634CE
- R1 = L0 ⊕ f(R0, K1) = C033C033 ⊕ 5B2634CE = 9B15F4FD
- L1 = R0 = 003C003C

Result after Round 1: L1 = 003C003C, R1 = 9B15F4FD

Step 4: Remaining Rounds (2-16)

The same Feistel procedure (expansion, XOR with round key, S-box substitution, P-permutation, XOR with the previous left half) is repeated for all 16 rounds, each using its own round key from Step 2. The resulting Li/Ri pairs are:

Round	**Li (hex)**	**Ri (hex)**
1	003C003C	9B15F4FD
2	9B15F4FD	0E293CBA
3	0E293CBA	ACEC6F38
4	ACEC6F38	07DCF0EE
5	07DCF0EE	1D7C0A9F
6	1D7C0A9F	05826A83
7	05826A83	A964DF29
8	A964DF29	EB8F2904
9	EB8F2904	B24403DF
10	B24403DF	62A3E77B
11	62A3E77B	B7ADD876
12	B7ADD876	C7198BCB
13	C7198BCB	D3EE3A89
14	D3EE3A89	BDAF90B8
15	BDAF90B8	9BEBAFFA
16	9BEBAFFA	EA5E2905

Step 5: Final Permutation (FP)

After round 16 there is no final swap of L16/R16; the pre-output block R16‖L16 is formed and passed through the FP table (the inverse of IP):

Pre-output block (R16 ‖ L16): EA5E29059BEBAFFA

Ciphertext after FP: ADFA19FE926E72EA

Final Answer — Ciphertext: ADFA19FE926E72EA

Question 3

Perform DES encryption for the following plaintext and key, showing the Initial Permutation, generation of the 16 round keys, the Round 1 Feistel operation, the Final Permutation, and the resulting ciphertext.

Plaintext (64-bit, hex): 123456ABCD132536

Key (64-bit, hex): AABBCCDDEEFF0011

Answer

Step 1: Initial Permutation (IP)

The 64-bit plaintext is rearranged according to the standard DES IP table, producing:

IP output: 14A7D67818CA18AD

L0 (left 32 bits): 14A7D678

R0 (right 32 bits): 18CA18AD

Step 2: Generation of the 16 Round Keys

The 64-bit key is first reduced to 56 bits via Permuted Choice 1 (PC-1), split into two 28-bit halves C0 and D0. Each half is left-circular-shifted by a fixed schedule (1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1 bits for rounds 1-16), and Permuted Choice 2 (PC-2) extracts the 48-bit round key K(i) from the concatenation of Ci and Di at each round.

Round	**Round Key K(i) — 48 bits (hex)**
1	365F93EB96A8
2	6E7B011BF6EA
3	0BBD7D3CDD25
4	CD64DB8A6CF6
5	77CFA8EDEB91
6	DAB983B3465B
7	39AE5FDF9306
8	65748E9467EC
9	5437EC557CC5
10	D2DC716AA0FD
11	CDEB66A3FD8F
12	A2F78F2E17B3
13	791763DF4967
14	E1D8F946CBD8
15	95E3D6D1B55D
16	B64F5EED965D

Step 3: Round 1 Feistel Operation

Round 1 takes L0 and R0 from the IP output and round key K1:

- R0 is expanded from 32 to 48 bits via the E-table: E(R0) = 8F16540F155A
- E(R0) is XORed with round key K1: E(R0) ⊕ K1 = B949C7E483F2
- The 48-bit result is split into eight 6-bit blocks and passed through S-boxes S1-S8, producing a 32-bit output: B205A9A6
- The S-box output is permuted through the P-table, giving f(R0, K1) = D18236A3
- R1 = L0 ⊕ f(R0, K1) = 14A7D678 ⊕ D18236A3 = C525E0DB
- L1 = R0 = 18CA18AD

Result after Round 1: L1 = 18CA18AD, R1 = C525E0DB

Step 4: Remaining Rounds (2-16)

The same Feistel procedure (expansion, XOR with round key, S-box substitution, P-permutation, XOR with the previous left half) is repeated for all 16 rounds, each using its own round key from Step 2. The resulting Li/Ri pairs are:

Round	Li (hex)	Ri (hex)
1	18CA18AD	C525E0DB
2	C525E0DB	303B8031
3	303B8031	FBFD165A
4	FBFD165A	BC51CA85
5	BC51CA85	6AB18016
6	6AB18016	FB0D8F58
7	FB0D8F58	5E79D34B
8	5E79D34B	AFC266CD
9	AFC266CD	7C5B2FB3
10	7C5B2FB3	07C0976B
11	07C0976B	5A22B92A
12	5A22B92A	060F6BCB
13	060F6BCB	1258D50E
14	1258D50E	3CBB4842
15	3CBB4842	07189441
16	07189441	0C09ED11

Step 5: Final Permutation (FP)

After round 16 there is no final swap of L16/R16; the pre-output block R16‖L16 is formed and passed through the FP table (the inverse of IP):

Pre-output block (R16 ‖ L16): 0C09ED1107189441

Ciphertext after FP: 9780CC742904060C

Final Answer — Ciphertext: 9780CC742904060C

Question 4

Perform DES encryption for the following plaintext and key, showing the Initial Permutation, generation of the 16 round keys, the Round 1 Feistel operation, the Final Permutation, and the resulting ciphertext.

Plaintext (64-bit, hex): FEDCBA9876543210

Key (64-bit, hex): 0F1571C947D9E859

Answer

Step 1: Initial Permutation (IP)

The 64-bit plaintext is rearranged according to the standard DES IP table, producing:

IP output: 33FF33000F550F55

L0 (left 32 bits): 33FF3300

R0 (right 32 bits): 0F550F55

Step 2: Generation of the 16 Round Keys

The 64-bit key is first reduced to 56 bits via Permuted Choice 1 (PC-1), split into two 28-bit halves C0 and D0. Each half is left-circular-shifted by a fixed schedule (1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1 bits for rounds 1-16), and Permuted Choice 2 (PC-2) extracts the 48-bit round key K(i) from the concatenation of Ci and Di at each round.

Round	Round Key K(i) — 48 bits (hex)
1	7833C320DA70
2	2B1A74CA48D8
3	8C78D881D31D
4	1667789316A0
5	CE5D01D80B25
6	4BAB4D126A9C
7	09F48B713191
8	710DEAA3202B
9	129AB83347C3
10	9C38661E8103
11	A26E4CC66544
12	48772468A3C8
13	C09D79F0D40B
14	C5E2634E162A
15	A3DF829C7968
16	A6120B4D4C25

Step 3: Round 1 Feistel Operation

Round 1 takes L0 and R0 from the IP output and round key K1:

- R0 is expanded from 32 to 48 bits via the E-table: E(R0) = 85EAAA85EAAA
- E(R0) is XORed with round key K1: E(R0) ⊕ K1 = FDD969A530DA
- The 48-bit result is split into eight 6-bit blocks and passed through S-boxes S1-S8, producing a 32-bit output: DBDA1100
- The S-box output is permuted through the P-table, giving f(R0, K1) = 64C9E142
- R1 = L0 ⊕ f(R0, K1) = 33FF3300 ⊕ 64C9E142 = 5736D242
- L1 = R0 = 0F550F55

Result after Round 1: L1 = 0F550F55, R1 = 5736D242

Step 4: Remaining Rounds (2-16)

The same Feistel procedure (expansion, XOR with round key, S-box substitution, P-permutation, XOR with the previous left half) is repeated for all 16 rounds, each using its own round key from Step 2. The resulting Li/Ri pairs are:

Round	Li (hex)	Ri (hex)
1	0F550F55	5736D242
2	5736D242	96C5CCE9
3	96C5CCE9	E3C7CCD3
4	E3C7CCD3	1B3C8FB9
5	1B3C8FB9	59D1D350
6	59D1D350	CAA1D647
7	CAA1D647	2ACABAFD
8	2ACABAFD	E251C0B7
9	E251C0B7	279D998F
10	279D998F	BFB43BF5
11	BFB43BF5	CF2990F3
12	CF2990F3	7264160A
13	7264160A	4A3F952E
14	4A3F952E	0D6D48A0
15	0D6D48A0	C07B5CE7
16	C07B5CE7	13912F18

Step 5: Final Permutation (FP)

After round 16 there is no final swap of L16/R16; the pre-output block R16‖L16 is formed and passed through the FP table (the inverse of IP):

Pre-output block (R16 ‖ L16): 13912F18C07B5CE7

Ciphertext after FP: 76660E2D7926AA92

Final Answer — Ciphertext: 76660E2D7926AA92

Question 5

Perform DES encryption for the following plaintext and key, showing the Initial Permutation, generation of the 16 round keys, the Round 1 Feistel operation, the Final Permutation, and the resulting ciphertext.

Plaintext (64-bit, hex): 02468ACEECA86420

Key (64-bit, hex): 0E329232EA6D0D73

Answer

Step 1: Initial Permutation (IP)

The 64-bit plaintext is rearranged according to the standard DES IP table, producing:

IP output: 5A005A003CF03C0F

L0 (left 32 bits): 5A005A00

R0 (right 32 bits): 3CF03C0F

Step 2: Generation of the 16 Round Keys

The 64-bit key is first reduced to 56 bits via Permuted Choice 1 (PC-1), split into two 28-bit halves C0 and D0. Each half is left-circular-shifted by a fixed schedule (1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1 bits for rounds 1-16), and Permuted Choice 2 (PC-2) extracts the 48-bit round key K(i) from the concatenation of Ci and Di at each round.

Round	Round Key K(i) — 48 bits (hex)
1	36146478E1E1
2	40BD1176E8FD
3	45A473239DDB
4	E7C4828FB533
5	7A83826F4F64
6	38901B58C9DE
7	25005EC5D49D
8	264894CB36E9
9	54554179F633
10	43C9453F4C2E
11	09E1878C79D6
12	3105ABA5E2F5
13	F100A1F38EC3
14	918A949E871F
15	1432961F77C4
16	606F044C3AE7

Step 3: Round 1 Feistel Operation

Round 1 takes L0 and R0 from the IP output and round key K1:

- R0 is expanded from 32 to 48 bits via the E-table: E(R0) = 9F97A01F805E
- E(R0) is XORed with round key K1: E(R0) ⊕ K1 = A983C46761BF
- The 48-bit result is split into eight 6-bit blocks and passed through S-boxes S1-S8, producing a 32-bit output: 6CAE3AEB
- The S-box output is permuted through the P-table, giving f(R0, K1) = 387A9FD5
- R1 = L0 ⊕ f(R0, K1) = 5A005A00 ⊕ 387A9FD5 = 627AC5D5
- L1 = R0 = 3CF03C0F

Result after Round 1: L1 = 3CF03C0F, R1 = 627AC5D5

Step 4: Remaining Rounds (2-16)

The same Feistel procedure (expansion, XOR with round key, S-box substitution, P-permutation, XOR with the previous left half) is repeated for all 16 rounds, each using its own round key from Step 2. The resulting Li/Ri pairs are:

Round	Li (hex)	Ri (hex)
1	3CF03C0F	627AC5D5
2	627AC5D5	F0DBB0DD
3	F0DBB0DD	750C1B2B
4	750C1B2B	EE54C3AA
5	EE54C3AA	CAB69490
6	CAB69490	E2B399FC
7	E2B399FC	CB1361F5
8	CB1361F5	41093AF3
9	41093AF3	1A41B740
10	1A41B740	827B996B
11	827B996B	9F810F04
12	9F810F04	F6AF9083
13	F6AF9083	D79C9E90
14	D79C9E90	6DB2A1DA
15	6DB2A1DA	D595898D
16	D595898D	69B3AD29

Step 5: Final Permutation (FP)

After round 16 there is no final swap of L16/R16; the pre-output block R16‖L16 is formed and passed through the FP table (the inverse of IP):

Pre-output block (R16 ‖ L16): 69B3AD29D595898D

Ciphertext after FP: FF10A64FB055C0BE

Final Answer — Ciphertext: FF10A64FB055C0BE

Question 6

Encrypt the following plaintext using the Blowfish algorithm, showing the initialization, key expansion, Feistel rounds, F-function computation, and the resulting ciphertext.

Plaintext (64-bit, hex): 0123456789ABCDEF

Key (128-bit, hex): FEDCBA98765432100123456789ABCDEF

Answer

Step 1: Initialization of the P-array and S-boxes

Blowfish's 18-entry P-array and four 256-entry S-boxes (S1-S4) are first initialized with a fixed string derived from the hexadecimal digits of pi. The first few standard values are:

P1, P2, P3, P4 (initial): 0x243F6A88, 0x85A308D3, 0x13198A2E, 0x03707344

S1[0..3] (initial): 0xD1310BA6, 0x98DFB5AC, 0x2FFD72DB, 0xD01ADFB7

(S2, S3, S4 are likewise initialized with further digits of pi; all 18 P-values and 1024 S-box values follow the published Blowfish specification.)

Step 2: Key Expansion

- The 128-bit key is split into 32-bit chunks and cyclically XORed into P1...P18 (wrapping around since the key is shorter than 18x32 bits).
- A 64-bit all-zero block is then repeatedly encrypted using the (partially key-dependent) Blowfish algorithm; each encryption's output replaces the next pair of P-array entries, then the next pairs of S-box entries, until all of P and S1-S4 have been replaced (521 total encryptions).

After key expansion, the resulting subkeys used for this question's encryption begin:

P1 (final, after key expansion): 0x33B7DFB4

P2 (final, after key expansion): 0xF66B3A0B

P3 (final, after key expansion): 0x30FA1DC8

Full final P-array (P1...P18):

P1	**P2**	**P3**	**P4**	**P5**	**P6**	**P7**	**P8**	**P9**
0x33B7DFB4	0xF66B3A0B	0x30FA1DC8	0x397F80D9	0x876EDDB2	0xAE70337B	0xADB5CD89	0xD810499F	0xF94DA4FB
P10	**P11**	**P12**	**P13**	**P14**	**P15**	**P16**	**P17**	**P18**
0x7408B570	0x6577AC43	0xA71E2203	0x904D8172	0xB8B37558	0x055F64D5	0x1376790F	0xEE9315D2	0x3B39BF15

Step 3: F-function Computation (illustrated for Round 1)

The 64-bit plaintext is split into L0 and R0:

L0: 0x01234567

R0: 0x89ABCDEF

- L0 is XORed with P1: L0 $\oplus$ P1 = 0x01234567 $\oplus$ 0x33B7DFB4 = 0x32949AD3
- This 32-bit value is split into four bytes: a=0x32, b=0x94, c=0x9A, d=0xD3
- Each byte indexes into S1, S2, S3, S4 respectively: S1[a]=0xD4A79FBE, S2[b]=0x6C6F4AE5, S3[c]=0x4C633171, S4[d]=0xD71D8667
- F = ((S1[a] + S2[b]) mod 2^{32}) $\oplus$ S3[c], then + S4[d] (mod 2^{32}):
- S1[a] + S2[b] = 0x4116EAA3
- (S1[a] + S2[b]) $\oplus$ S3[c] = 0x0D75DBD2
- F(L0) = 0xE4936239
- R0 is then updated: R0 $\oplus$ F(L0) = 0x6D38AFD6, after which L and R are swapped to begin Round 2.

Step 4: All 16 Feistel Rounds

Each of the 16 rounds XORs the left half with the next P-array entry (P1...P16), applies the F-function to feed into the right half, then swaps L and R. The L/R values after each round are:

Round	L (hex)	R (hex)
1	0x6D38AFD6	0x32949AD3
2	0x3B3F2FC3	0x9B5395DD
3	0xF13033FA	0x0BC5320B
4	0x9709A5D2	0xC84FB323
5	0xCBAC237F	0x10677860
6	0x09C7B7BD	0x65DC1004
7	0xE7871C6E	0xA4727A34
8	0x9193309C	0x3F9755F1
9	0xE5BF6D99	0x68DE9467
10	0xFAEF5457	0x91B7D8E9
11	0xDBE7045D	0x9F98F814
12	0x47792FF4	0x7CF9265E
13	0x54F728C8	0xD734AE86
14	0xD9BB34AD	0xEC445D90
15	0xA924D7D8	0xDCE45078
16	0xE46F2522	0xBA52AED7

Step 5: Post-processing and Ciphertext

After the 16th round, the last swap is undone, and the halves are XORed with the two remaining P-array entries: R ⊕= P17, L ⊕= P18. The two 32-bit halves are then concatenated to form the 64-bit ciphertext.

Final Answer — Ciphertext: 816B11C20AFC30F0

Question 7

Encrypt the following plaintext using the Blowfish algorithm, showing the initialization, key expansion, Feistel rounds, F-function computation, and the resulting ciphertext.

Plaintext (64-bit, hex): 1234567890ABCDEF

Key (128-bit, hex): 00112233445566778899AABBCCDDEEFF

Answer

Step 1: Initialization of the P-array and S-boxes

Blowfish's 18-entry P-array and four 256-entry S-boxes (S1-S4) are first initialized with a fixed string derived from the hexadecimal digits of pi. The first few standard values are:

P1, P2, P3, P4 (initial): 0x243F6A88, 0x85A308D3, 0x13198A2E, 0x03707344

S1[0..3] (initial): 0xD1310BA6, 0x98DFB5AC, 0x2FFD72DB, 0xD01ADFB7

(S2, S3, S4 are likewise initialized with further digits of pi; all 18 P-values and 1024 S-box values follow the published Blowfish specification.)

Step 2: Key Expansion

- The 128-bit key is split into 32-bit chunks and cyclically XORed into P1...P18 (wrapping around since the key is shorter than 18x32 bits).
- A 64-bit all-zero block is then repeatedly encrypted using the (partially key-dependent) Blowfish algorithm; each encryption's output replaces the next pair of P-array entries, then the next pairs of S-box entries, until all of P and S1-S4 have been replaced (521 total encryptions).

After key expansion, the resulting subkeys used for this question's encryption begin:

P1 (final, after key expansion): 0xFAC3397F

P2 (final, after key expansion): 0xEC61D9ED

P3 (final, after key expansion): 0xD1FF7D60

Full final P-array (P1...P18):

P1	**P2**	**P3**	**P4**	**P5**	**P6**	**P7**	**P8**	**P9**
0xFAC3397F	0xEC61D9ED	0xD1FF7D60	0xA146C4D9	0x5237D80F	0x9C341EFE	0x2679565A	0x8AFF61B2	0xCE008F22
P10	**P11**	**P12**	**P13**	**P14**	**P15**	**P16**	**P17**	**P18**
0xC62B1FE1	0x841F773D	0xBA7053F1	0xA04C51B1	0x22AF94D8	0xB6581859	0xBB55B2F3	0x63FE08DA	0xACEB779B

Step 3: F-function Computation (illustrated for Round 1)

The 64-bit plaintext is split into L0 and R0:

L0: 0x12345678

R0: 0x90ABCDEF

- L0 is XORed with P1: L0 ⊕ P1 = 0x12345678 ⊕ 0xFAC3397F = 0xE8F76F07
- This 32-bit value is split into four bytes: a=0xE8, b=0xF7, c=0x6F, d=0x07
- Each byte indexes into S1, S2, S3, S4 respectively: S1[a]=0x750B9070, S2[b]=0x1CD16DC5, S3[c]=0xC672DA5D, S4[d]=0x5E49A1C3
- F = ((S1[a] + S2[b]) mod 2^32) ⊕ S3[c], then + S4[d] (mod 2^32):
- S1[a] + S2[b] = 0x91DCFE35
- (S1[a] + S2[b]) ⊕ S3[c] = 0x57AE2468
- F(L0) = 0xB5F7C62B
- R0 is then updated: R0 ⊕ F(L0) = 0x255C0BC4, after which L and R are swapped to begin Round 2.

Step 4: All 16 Feistel Rounds

Each of the 16 rounds XORs the left half with the next P-array entry (P1...P16), applies the F-function to feed into the right half, then swaps L and R. The L/R values after each round are:

Round	L (hex)	R (hex)
1	0x255C0BC4	0xE8F76F07
2	0xDF9BD74C	0xC93DD229
3	0x9C84B9E8	0x0E64AA2C
4	0xDD8230B4	0x3DC27D31
5	0xCC4A8397	0x8FB5E8BB
6	0x04CF98B4	0x507E9D69
7	0xC3882093	0x22B6CEEE
8	0x4245307D	0x49774121
9	0x39B7842A	0x8C45BF5F
10	0x2FFC25FD	0xFF9C9BCB
11	0x7BAB92C2	0xABE352C0
12	0x055E07F5	0xC1DBC133
13	0xB2CF6142	0xA5125644
14	0x0EA6CC80	0x9060F59A
15	0x925B699C	0xB8FED4D9
16	0x666E6BAB	0x290EDB6F

Step 5: Post-processing and Ciphertext

After the 16th round, the last swap is undone, and the halves are XORed with the two remaining P-array entries: R ⊕= P17, L ⊕= P18. The two 32-bit halves are then concatenated to form the 64-bit ciphertext.

Final Answer — Ciphertext: 85E5ACF405906371

Question 8

Encrypt the following plaintext using the Blowfish algorithm, showing the initialization, key expansion, Feistel rounds, F-function computation, and the resulting ciphertext.

Plaintext (64-bit, hex): FEDCBA9876543210

Key (128-bit, hex): AABBCCDDEEFF00112233445566778899

Answer

Step 1: Initialization of the P-array and S-boxes

Blowfish's 18-entry P-array and four 256-entry S-boxes (S1-S4) are first initialized with a fixed string derived from the hexadecimal digits of pi. The first few standard values are:

P1, P2, P3, P4 (initial): 0x243F6A88, 0x85A308D3, 0x13198A2E, 0x03707344

S1[0..3] (initial): 0xD1310BA6, 0x98DFB5AC, 0x2FFD72DB, 0xD01ADFB7

(S2, S3, S4 are likewise initialized with further digits of pi; all 18 P-values and 1024 S-box values follow the published Blowfish specification.)

Step 2: Key Expansion

- The 128-bit key is split into 32-bit chunks and cyclically XORed into P1...P18 (wrapping around since the key is shorter than 18x32 bits).
- A 64-bit all-zero block is then repeatedly encrypted using the (partially key-dependent) Blowfish algorithm; each encryption's output replaces the next pair of P-array entries, then the next pairs of S-box entries, until all of P and S1-S4 have been replaced (521 total encryptions).

After key expansion, the resulting subkeys used for this question's encryption begin:

P1 (final, after key expansion): 0x1294B769

P2 (final, after key expansion): 0x7684BF25

P3 (final, after key expansion): 0x69F8BFED

Full final P-array (P1...P18):

P1	**P2**	**P3**	**P4**	**P5**	**P6**	**P7**	**P8**	**P9**
0x1294B769	0x7684BF25	0x69F8BFED	0x28104C00	0x15772BC3	0x8088BBA4	0x7D21D1DB	0xD052ED3A	0x682C4DC9
P10	**P11**	**P12**	**P13**	**P14**	**P15**	**P16**	**P17**	**P18**
0xBDABD0D6	0xA3AB9D72	0x140940EF	0x17222935	0xB0396F70	0x86BA5932	0x3330C5A9	0x124966A5	0x31EE4FD5

Step 3: F-function Computation (illustrated for Round 1)

The 64-bit plaintext is split into L0 and R0:

L0: 0xFEDCBA98

R0: 0x76543210

- L0 is XORed with P1: L0 ⊕ P1 = 0xFEDCBA98 ⊕ 0x1294B769 = 0xEC480DF1
- This 32-bit value is split into four bytes: a=0xEC, b=0x48, c=0x0D, d=0xF1
- Each byte indexes into S1, S2, S3, S4 respectively: S1[a]=0x803EBC10, S2[b]=0x71927CC2, S3[c]=0xD5033418, S4[d]=0x4B9AF547
- F = ((S1[a] + S2[b]) mod 2^32) ⊕ S3[c], then + S4[d] (mod 2^32):
- S1[a] + S2[b] = 0xF1D138D2
- (S1[a] + S2[b]) ⊕ S3[c] = 0x24D20CCA
- F(L0) = 0x706D0211
- R0 is then updated: R0 ⊕ F(L0) = 0x06393001, after which L and R are swapped to begin Round 2.

Step 4: All 16 Feistel Rounds

Each of the 16 rounds XORs the left half with the next P-array entry (P1...P16), applies the F-function to feed into the right half, then swaps L and R. The L/R values after each round are:

Round	L (hex)	R (hex)
1	0x06393001	0xEC480DF1
2	0x4383F3D0	0x70BD8F24
3	0xA3049071	0x2A7B4C3D
4	0x0ADCD2FC	0x8B14DC71
5	0x4F77FECF	0x1FABF93F
6	0x9E161597	0xCFFF456B
7	0x61487483	0xE337C44C
8	0x837A334D	0xB11A99B9
9	0x8CBAF0AB	0xEB567E84
10	0xEE636ADF	0x3111207D
11	0xDFB8AF0D	0x4DC8F7AD
12	0x2E22DA98	0xCBB1EFE2
13	0x0532B589	0x3900F3AD
14	0x6523042B	0xB50BDAF9
15	0xCFE34E72	0xE3995D19
16	0xFD74EE6C	0xFCD38BDB

Step 5: Post-processing and Ciphertext

After the 16th round, the last swap is undone, and the halves are XORed with the two remaining P-array entries: R ⊕= P17, L ⊕= P18. The two 32-bit halves are then concatenated to form the 64-bit ciphertext.

Final Answer — Ciphertext: CD3DC40EEF3D88C9

Question 9

Encrypt the following plaintext using the Blowfish algorithm, showing the initialization, key expansion, Feistel rounds, F-function computation, and the resulting ciphertext.

Plaintext (64-bit, hex): 89ABCDEF01234567

Key (128-bit, hex): 1234567890ABCDEFFEDCBA0987654321

Answer

Step 1: Initialization of the P-array and S-boxes

Blowfish's 18-entry P-array and four 256-entry S-boxes (S1-S4) are first initialized with a fixed string derived from the hexadecimal digits of pi. The first few standard values are:

P1, P2, P3, P4 (initial): 0x243F6A88, 0x85A308D3, 0x13198A2E, 0x03707344

S1[0..3] (initial): 0xD1310BA6, 0x98DFB5AC, 0x2FFD72DB, 0xD01ADFB7

(S2, S3, S4 are likewise initialized with further digits of pi; all 18 P-values and 1024 S-box values follow the published Blowfish specification.)

Step 2: Key Expansion

- The 128-bit key is split into 32-bit chunks and cyclically XORed into P1...P18 (wrapping around since the key is shorter than 18x32 bits).
- A 64-bit all-zero block is then repeatedly encrypted using the (partially key-dependent) Blowfish algorithm; each encryption's output replaces the next pair of P-array entries, then the next pairs of S-box entries, until all of P and S1-S4 have been replaced (521 total encryptions).

After key expansion, the resulting subkeys used for this question's encryption begin:

P1 (final, after key expansion): 0x1DB804D4

P2 (final, after key expansion): 0x8571A980

P3 (final, after key expansion): 0x41CD82E9

Full final P-array (P1...P18):

P1	**P2**	**P3**	**P4**	**P5**	**P6**	**P7**	**P8**	**P9**
0x1DB804D4	0x8571A980	0x41CD82E9	0x611F1BA9	0xB22D6AA8	0x83637845	0xF410A430	0x99682F35	0x94CF9151
P10	**P11**	**P12**	**P13**	**P14**	**P15**	**P16**	**P17**	**P18**
0x3EF493A9	0x097F4A68	0x27944CD3	0xDDFEC982	0x6F6A11BA	0xEC54AE66	0xF0840B7F	0x7EF09B42	0xEBFE5D7D

Step 3: F-function Computation (illustrated for Round 1)

The 64-bit plaintext is split into L0 and R0:

L0: 0x89ABCDEF

R0: 0x01234567

- L0 is XORed with P1: L0 $\oplus$ P1 = 0x89ABCDEF $\oplus$ 0x1DB804D4 = 0x9413C93B
- This 32-bit value is split into four bytes: a=0x94, b=0x13, c=0xC9, d=0x3B
- Each byte indexes into S1, S2, S3, S4 respectively: S1[a]=0xEED8743F, S2[b]=0x59CF6920, S3[c]=0xD0329337, S4[d]=0xA8494FF8
- F = ((S1[a] + S2[b]) mod 2^{32}) $\oplus$ S3[c], then + S4[d] (mod 2^{32}):
- S1[a] + S2[b] = 0x48A7DD5F
- (S1[a] + S2[b]) $\oplus$ S3[c] = 0x98954E68
- F(L0) = 0x40DE9E60
- R0 is then updated: R0 $\oplus$ F(L0) = 0x41FDDB07, after which L and R are swapped to begin Round 2.

Step 4: All 16 Feistel Rounds

Each of the 16 rounds XORs the left half with the next P-array entry (P1...P16), applies the F-function to feed into the right half, then swaps L and R. The L/R values after each round are:

Round	L (hex)	R (hex)
1	0x41FDDB07	0x9413C93B
2	0x893CBC0B	0xC48C7287
3	0xF9EA5547	0xC8F13EE2
4	0x696D79F1	0x98F54EEE
5	0xA1EC9CAB	0xDB401359
6	0x46ABF138	0x228FE4EE
7	0xCCFD8901	0xB2BB5508
8	0x1A688A7C	0x5595A634
9	0x0F3985FA	0x8EA71B2D
10	0x9F525D88	0x31CD1653
11	0x29804362	0x962D17E0
12	0xC1D860E9	0x0E140FB1
13	0xADACBE83	0x1C26A96B
14	0x9E1EB0A3	0xC2C6AF39
15	0x64836EAD	0x724A1EC5
16	0x3A122285	0x940765D2

Step 5: Post-processing and Ciphertext

After the 16th round, the last swap is undone, and the halves are XORed with the two remaining P-array entries: R ⊕= P17, L ⊕= P18. The two 32-bit halves are then concatenated to form the 64-bit ciphertext.

Final Answer — Ciphertext: 7FF938AF44E2B9C7

Question 10

Encrypt the following plaintext using the Blowfish algorithm, showing the initialization, key expansion, Feistel rounds, F-function computation, and the resulting ciphertext.

Plaintext (64-bit, hex): ABCDEF1234567890

Key (128-bit, hex): 0F1E2D3C4B5A69788796A5B4C3D2E1F0

Answer

Step 1: Initialization of the P-array and S-boxes

Blowfish's 18-entry P-array and four 256-entry S-boxes (S1-S4) are first initialized with a fixed string derived from the hexadecimal digits of pi. The first few standard values are:

P1, P2, P3, P4 (initial): 0x243F6A88, 0x85A308D3, 0x13198A2E, 0x03707344

S1[0..3] (initial): 0xD1310BA6, 0x98DFB5AC, 0x2FFD72DB, 0xD01ADFB7

(S2, S3, S4 are likewise initialized with further digits of pi; all 18 P-values and 1024 S-box values follow the published Blowfish specification.)

Step 2: Key Expansion

- The 128-bit key is split into 32-bit chunks and cyclically XORed into P1...P18 (wrapping around since the key is shorter than 18x32 bits).
- A 64-bit all-zero block is then repeatedly encrypted using the (partially key-dependent) Blowfish algorithm; each encryption's output replaces the next pair of P-array entries, then the next pairs of S-box entries, until all of P and S1-S4 have been replaced (521 total encryptions).

After key expansion, the resulting subkeys used for this question's encryption begin:

P1 (final, after key expansion): 0x56E2C39E

P2 (final, after key expansion): 0x58FE0599

P3 (final, after key expansion): 0xF51D2B07

Full final P-array (P1...P18):

P1	**P2**	**P3**	**P4**	**P5**	**P6**	**P7**	**P8**	**P9**
0x56E2C39E	0x58FE0599	0xF51D2B07	0x0131D9F0	0x126A6D90	0xAB581BF8	0x2D24EF99	0x4D322897	0x1BD0C014
P10	**P11**	**P12**	**P13**	**P14**	**P15**	**P16**	**P17**	**P18**
0xF3D45DB1	0x37C2DDE3	0xA0652884	0x0D5151C7	0x75ADA50E	0xE1AE6A7A	0xFD9EA6A6	0x1F2A2AE6	0x00551B26

Step 3: F-function Computation (illustrated for Round 1)

The 64-bit plaintext is split into L0 and R0:

L0: 0xABCDEF12

R0: 0x34567890

- L0 is XORed with P1: L0 ⊕ P1 = 0xABCDEF12 ⊕ 0x56E2C39E = 0xFD2F2C8C
- This 32-bit value is split into four bytes: a=0xFD, b=0x2F, c=0x2C, d=0x8C
- Each byte indexes into S1, S2, S3, S4 respectively: S1[a]=0x134D5BE4, S2[b]=0x930509D3, S3[c]=0xB8688914, S4[d]=0xF56B49F7
- F = ((S1[a] + S2[b]) mod 2^32) ⊕ S3[c], then + S4[d] (mod 2^32):
- S1[a] + S2[b] = 0xA65265B7
- (S1[a] + S2[b]) ⊕ S3[c] = 0x1E3AECA3
- F(L0) = 0x13A6369A
- R0 is then updated: R0 ⊕ F(L0) = 0x27F04E0A, after which L and R are swapped to begin Round 2.

Step 4: All 16 Feistel Rounds

Each of the 16 rounds XORs the left half with the next P-array entry (P1...P16), applies the F-function to feed into the right half, then swaps L and R. The L/R values after each round are:

Round	L (hex)	R (hex)
1	0x27F04E0A	0xFD2F2C8C
2	0x1E5EBC51	0x7F0E4B93
3	0xC44E4ABC	0xEB439756
4	0xF0BA97AC	0xC57F934C
5	0xDD7C0508	0xE2D0FA3C
6	0xDB5B44BF	0x76241EF0
7	0x9A54BB48	0xF67FAB26
8	0xF0A97677	0xD76693DF
9	0xFE6E5AD2	0xEB79B663
10	0x0AF38BCB	0x0DBA0763
11	0x9D3A2F0D	0x3D315628
12	0x9E3A1F70	0x3D5F0789
13	0x5F228CAC	0x936B4EB7
14	0x5F9AA342	0x2A8F29A2
15	0x255DE5E6	0xBE34C938
16	0x057DF1FF	0xD8C34340

Step 5: Post-processing and Ciphertext

After the 16th round, the last swap is undone, and the halves are XORed with the two remaining P-array entries: R ⊕= P17, L ⊕= P18. The two 32-bit halves are then concatenated to form the 64-bit ciphertext.

Final Answer — Ciphertext: D89658661A57DB19